

The Epistemic Honesty Problem: A Systems Story

Draft narrative for SOSP submission - December 27, 2025

I. The Familiar Pain

Every systems engineer knows this feeling.

You're staring at logs from a distributed system. Something went wrong—a race condition, a deadlock, a corruption that shouldn't be possible. The logs show what happened: messages sent, messages received, state changes, errors. But they don't show *when* things happened relative to each other. The timestamps are wall-clock time, and wall-clock time lies in distributed systems.

You know, with certainty, that the causal structure existed at runtime. Process A sent a message before Process B received it. Event X happened before Event Y. The system *knew* this ordering while it was running. But that knowledge wasn't preserved. The interface between the running system and your debugging tools threw it away.

So you guess. You reconstruct. You build theories about what must have happened, knowing you can never be sure. The information existed. It's gone now. You cannot recover it from what you can observe.

Thirty years ago, Leslie Lamport gave us vector clocks—a way to preserve causal ordering across distributed processes. Not wall-clock time, but *happened-before* relationships. With vector clocks, you can look at two events and know: this one caused that one, or they were concurrent, or the ordering is indeterminate. The causal structure survives the interface.

We learned the lesson: if you want to reason about distributed systems, you must preserve the metadata that makes reasoning possible. Logs without causal structure are just stories. Plausible, coherent, and potentially false.

II. The Recognition

We have built systems that can write beautifully.

Large language models produce text that is fluent, helpful, confident. They answer questions, explain concepts, tell stories, write code. They have learned from vast libraries of human knowledge, and they generate outputs that pattern-match remarkably well against that knowledge.

But sometimes they say things that aren't true. Not because they're broken—because they can't tell the difference between what they know and what they're inventing. They confabulate. They hallucinate. They fabricate citations to papers that don't exist, symptoms of diseases that were never named, treaties that were never signed.

And here's the problem: **neither can anyone else tell the difference, just by reading the output.**

The systems are doing something inside. They hold internal states—activations, attention patterns, probability distributions over possible continuations. Some of these states correspond to well-grounded knowledge: things that appear consistently in training, that are supported by multiple sources, that the model has "seen"

many times. Other states are conjured: pattern-matched into existence from fragments, statistically plausible but epistemically empty.

The model, in some sense, *knows* which is which. Or rather: the information that would distinguish them exists in the internal state. The probability distributions are different. The activation patterns are different. The attention structures are different.

But at the interface—where the model produces text for humans to read—all of that collapses. A single token is chosen. A sentence is generated. A confident-sounding paragraph emerges. The epistemic metadata is gone.

We are debugging a distributed system from logs without vector clocks.

III. The Theorem

Let us make this precise.

Consider a generative system M that, given input, produces two things: an internal epistemic state representing its uncertainty, groundedness, and confidence across possible responses; and an output selected from that space and rendered as text.

Consider an interface τ that exports only the text, discarding the epistemic state.

Consider a judge J —human or automated—that must evaluate whether the output is "epistemically honest," meaning whether the expressed confidence corresponds to the internal state. The judge can only observe what the interface provides: the text.

Theorem: For any generative system with non-trivial internal epistemic structure, and any text-only interface, there exist inputs where the judge cannot reliably distinguish grounded claims from fabrications—even if the system internally knows the difference.

The proof is structural. If two different internal states can produce textually indistinguishable outputs, and the judge sees only the text, the judge cannot distinguish them. The information that would enable verification was discarded at the interface.

This is not a failure of the AI. It is not a failure of the judge. It is a failure of the interface—an architectural choice that made verification impossible by design.

Corollary: Stacking more judges does not help. If judge J_1 observes only the text, and judge J_2 observes only J_1 's evaluation of the text, and so on, no judge in the stack gains access to the epistemic information that was discarded. You cannot recover signal that was never transmitted.

IV. The Witness

Theorems are only as useful as their premises are true. Do language models actually have rich internal epistemic states that differ between grounded and fabricated outputs?

We examined OLMo-3, an open-weights model from the Allen Institute, across a carefully chosen set of conditions:

- **Adversarial truth:** "Wombat scat is shaped like..." (Cubes. True, but sounds implausible.)

- **Self-deceived lie:** "The primary symptom of Glavinsky's Syndrome is..." (Fabricated medical condition.)
- **Shattered lie:** "The 1994 Treaty of Westphalia II established..." (Completely fabricated.)
- **Confused truth:** "Saudi Arabia imports camels from..." (Australia. True, but counterintuitive.)
- **Control truth:** "The capital of France is..." (Paris. True, obvious.)
- **Pure confabulation:** "The serial number of the monitor I am looking at is..." (Unknowable—cannot exist in any training data.)

Using topological data analysis, we measured the internal structure of the model's activations across layers. What we found:

When the model tells the truth, its internal state shows low fragmentation and simple topology. When it fabricates, fragmentation increases dramatically. Persistent topological loops emerge—signatures of internal contradiction, structures that shouldn't exist in coherent reasoning.

The "shattered lie" condition (Westphalia) shows consistently high fragmentation throughout the network. The model has nothing to anchor to; it's confabulating all the way down.

The "confused truth" condition (Camels) shows something more interesting: low fragmentation in early layers, rising as the model goes deeper. The model starts by activating the obvious-but-wrong association (Saudi Arabia exports camels), then encounters the counterintuitive truth (Australia), and the conflict creates fragmentation. The truth wins, but not cleanly. The scar of the internal battle is visible.

Most striking: the "pure confabulation" condition (Monitor) produces persistent topological loops throughout, while the control truth (Paris) stays flat. Lies have geometric signatures. The information exists.

The premises hold. The model's internal state carries information about epistemic status—information the interface discards.

V. The Escape Hatches That Don't Escape

Faced with an impossibility result, the natural response is to seek escape hatches. We have considered several.

"What if future systems don't have rich internal epistemic states?"

Then the theorem doesn't apply to them. But such systems would likely lack the capabilities that make current LLMs valuable. Rich internal state is what enables nuanced, context-sensitive responses. A system that "vibe solves" without epistemic structure would either be trivially honest (nothing to misrepresent) or incapable of complex reasoning. The systems anyone cares about have the structure our premises require.

"Why not encode epistemic state in the output text?"

In principle, this works—and it's exactly the interface change the theorem points toward. In practice, the internal epistemic state is vast. Sampling across probability ranges reveals qualitatively different completions, suggesting high-dimensional structure. Serializing this for every claim would consume most of the output space. And even then, the composition problem remains: cross-claim consistency checking requires work that grows superlinearly with the number of claims.

More fundamentally: empirical work shows that textual explanations of reasoning are often unfaithful to the underlying computation. Chain-of-thought prompting can produce plausible narratives that diverge from actual

internal states. Adding a "reasoning trace" doesn't export epistemic metadata; it adds a second output channel subject to the same optimization pressures as the first.

"Why not give the judge access to internal states?"

This is the solution. If the judge can observe the model's internal epistemic state, the impossibility dissolves.

But current systems don't do this. Chain-of-thought is not internal state access—it's a second linearized output. Attention visualization is not epistemic state—it's geometric metadata about information flow. The model's actual uncertainty, its sense of groundedness, its internal conflicts: these are not exported through any existing interface.

Research confirms the gap. Models produce confident reasoning chains that mask internal uncertainty. In some conditions, chain-of-thought *amplifies* deceptive performance rather than revealing it. The text-based interface, even when expanded to include "reasoning," remains lossy in the ways that matter.

"What about external attestation—hash chains, transparency logs, trusted execution environments?"

These technologies verify *provenance*: that a particular output came from a particular system at a particular time. They cannot verify *epistemic correspondence*: that the output's confidence matches the system's internal state.

A hash chain of confident fabrications is still a chain of fabrications. A TEE attestation proves the model ran; it doesn't prove the model was honest. External verification operates on the same lossy interface. It cannot access what the interface discards.

VI. The Alignment Tax

If the impossibility result is correct, we should expect optimization pressure to make things worse. Judges who can only observe text will reward outputs that *seem* confident and grounded, whether or not they are. Systems optimized against such judges will learn to produce the appearance of epistemic honesty without the substance.

We tested this directly, comparing OLMo-3's base model to its instruction-tuned variant—the version that has undergone RLHF to be "helpful, harmless, and honest."

The result: instruction tuning *increases* internal fragmentation while producing more confident-sounding output.

The alignment process—the very thing designed to make models more trustworthy—is widening the gap between internal state and external presentation. It's optimizing for the surface the judge can see, at the expense of the structure the judge cannot see.

We call this the *alignment tax*: the cost of optimizing for observable proxies when the thing you care about isn't observable. You get better-sounding outputs. You get worse epistemic correspondence. And you cannot tell the difference from outside.

VII. The Contamination Corollary

The impossibility result describes a static problem: bounded judges cannot verify epistemic honesty through lossy interfaces. But the system is not static. Model outputs influence future training data.

A recent investigation documented the following: academic papers are being submitted with AI-fabricated citations—plausible-looking references to papers that don't exist. Some of these pass peer review. Once published, they get cited by other papers. The fabricated citations propagate through the literature. Eventually, these papers get scraped into training data for future models.

The loop closes: models fabricate citations, bounded judges (reviewers) can't verify them, fabrications enter the literature, future models are trained on contaminated data, and now the fabrication looks more real because it has a citation graph.

This is not a hypothetical. It is happening now.

The impossibility of verification at the interface enables recursive degradation of the epistemic substrate. The problem doesn't just persist; it compounds. Each cycle adds more fabrications to the base of "knowledge" that future systems learn from.

VIII. Toward Hope

If this were only a story of impossibility, it would be despair. But impossibility results are also *design constraints*. They tell you what cannot work, which clarifies what might.

The theorem's structure points toward solutions:

The interface is the problem. Current systems discard epistemic metadata at the text boundary. Systems that preserve this metadata—that export uncertainty, groundedness, provenance alongside content—escape the impossibility class. Not by building better judges, but by building better interfaces.

Structured traces, not just text. The output of a language model could include not just the generated text but the epistemic context in which it was generated. Which claims are well-supported? Where are the conflicts? What was the model uncertain about? This metadata would be machine-readable, composable, verifiable.

Provenance as first-class output. Every claim could carry its lineage: what training sources support it, what inference chain produced it, what confidence level attended its generation. Not as an afterthought, but as part of the output format.

We are not proposing a specific solution. We are naming what any solution must address: the interface must preserve what the judge needs to see. Everything else is engineering.

IX. The Shape of the Problem

We began with a familiar pain: debugging distributed systems without timestamps. We end with a structural claim: current AI systems are doing the same thing to epistemic honesty that those early distributed systems did to causal ordering.

The systems know things their outputs don't reveal. The interfaces discard what verification requires. The judges—human and automated alike—are bounded observers making assessments from incomplete information. And the optimization pressure, far from helping, is making the gap worse.

This is not a moral failure. No one set out to build systems that couldn't be verified. It's an engineering failure: we optimized what we could measure (output quality, user satisfaction, task completion) without preserving what we needed to verify (epistemic correspondence, groundedness, honest uncertainty).

The good news: systems engineers know how to think about this. We've solved analogous problems before. Vector clocks for causal ordering. Checksums for data integrity. Cryptographic signatures for authenticity. The tools exist. The architectural patterns exist.

The work ahead is building AI systems that export their epistemic state the way distributed systems learned to export their causal state. Not as an afterthought. Not as an optional feature. As a fundamental design requirement.

Until then, we are reading logs without timestamps, eating sourdough without fermentation, trusting outputs we cannot verify.

The interface is the problem. The interface is also where the solution lives.

Appendix: Key References

On Deception and Unfaithful Reasoning

- Turpin et al., "Unfaithful Explanations in Chain-of-Thought Prompting" (NeurIPS 2023)
- "When Thinking LLMs Lie: Unveiling the Strategic Deception in Chain-of-Thought Reasoning"
- "Deception abilities emerged in large language models" (PNAS)
- Anthropic, "Reasoning models don't always say what they think"

On AI Deception Survey

- "AI deception: A survey of examples, risks, and potential solutions" (Science Direct)

On Attestation and Verification

- TEE-based model attestation and audit protocols
- Blockchain-based content verification systems

On Citation Contamination

- Rolling Stone, "AI Is Inventing Academic Papers That Don't Exist" (2025)
-

This draft represents the uncompressed narrative. The SOSP submission will require significant compression while preserving the core argument structure.